

The Complete SQL Encyclopedia & Exam Bible

สารานุกรมรวมคำสั่ง SQL ทุกหมวดหมู่ ทุกฟังก์ชัน ทุก Operators พร้อมแนวข้อสอบจริง 25 ข้อละเอียดที่สุด

DDL / DML / DQL / TCL / DCL

Scalar & Aggregate Functions

Relational Joins & Subqueries

CTEs & Window Functions

Views, Triggers & ACID

25 Real-World Problems

หมวดที่ 1: DDL (Data Definition Language) – การจัดการโครงสร้างตาราง

คำสั่ง DDL ใช้สำหรับกำหนดโครงสร้าง นิยามตาราง ชนิดข้อมูล และ Constraints ทั้งหมดในฐานข้อมูล

1.1 การสร้างและลบ Database / Table

```
-- สร้างฐานข้อมูล
CREATE DATABASE IF NOT EXISTS
db_water
CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;

-- เลือกลูกคำ
USE db_water;

-- ลบตารางทิ้งทั้งโครงสร้าง
DROP TABLE IF EXISTS old_table;
```

1.2 สรุปชนิดข้อมูลสำคัญ (Data Types)

- INT / BIGINT : ตัวเลขจำนวนเต็ม
- DECIMAL(10,2) : ตัวเลขเงิน/ทศนิยมแม่นยำ ไม่ปัดเศษเพี้ยน
- VARCHAR(N) : ตัวหนังสือแปรผันตามความยาวจริง
- CHAR(N) : ตัวหนังสือความยาวคงที่ (เช่น รหัสไปรษณีย์)
- DATE / DATETIME / TIMESTAMP : วันที่และเวลา

1.3 ตัวอย่าง CREATE TABLE พร้อม Constraints ครบทุกตัว

```
CREATE TABLE tb_customers (
  cus_id INT(4) UNSIGNED ZEROFILL AUTO_INCREMENT, -- รันเลข 0001 อัตโนมัติ
  customer_code VARCHAR(20) NOT NULL UNIQUE, -- ห้ามว่าง และห้ามซ้ำ
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  zone_id INT NOT NULL, -- คีย์นอกเชื่อมโยง
  meter_size VARCHAR(20) DEFAULT 'มิเตอร์ 5/8 นิ้ว', -- ค่าตั้งต้น
  credit_limit DECIMAL(10,2) CHECK (credit_limit ≥ 0), -- เงินใจต้อง ≥ 0
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (cus_id),
  CONSTRAINT fk_cust_zone FOREIGN KEY (zone_id) REFERENCES tb_zone(zone_id)
  ON UPDATE CASCADE ON DELETE RESTRICT
);
```

1.4 การปรับแต่งโครงสร้างด้วย ALTER TABLE

```
ALTER TABLE tb_customers ADD COLUMN phone VARCHAR(15) AFTER last_name; -- เพิ่มคอลัมน์
ALTER TABLE tb_customers DROP COLUMN credit_limit; -- ลบ
คอลัมน์
ALTER TABLE tb_customers MODIFY COLUMN phone VARCHAR(20) NOT NULL; -- แก้ไขชนิด
ข้อมูล
ALTER TABLE tb_customers CHANGE COLUMN phone cus_tel VARCHAR(20); -- เปลี่ยน
ชื่อคอลัมน์
ALTER TABLE tb_customers RENAME TO members; -- เปลี่ยน
ชื่อตาราง
```

1.5 ศึกษาเปรียบเทียบ: DROP vs TRUNCATE vs DELETE (ออกข้อสอบบ่อยมาก)

คำสั่ง	ประเภท	สิ่งที่เกิดขึ้น	ความเร็ว	Rollback ได้ไหม?	AUTO_INCREMENT
DROP TABLE	DDL	ลบทั้งโครงสร้างและข้อมูล ทั้งหมด ตารางหายไปเลย	เร็วมาก	✗ ไม่ได้	ตารางหายไป
TRUNCATE TABLE	DDL	ลบเฉพาะข้อมูลทั้งหมด โครงสร้างยังอยู่ครบ	เร็วมาก	✗ ไม่ได้	✓ รีเซ็ตกลับเป็น 1
DELETE FROM	DML	ลบข้อมูลเฉพาะแถวตามเงื่อนไข WHERE	ช้ากว่า	✓ ได้ (ใน Transaction)	✗ นับต่อจากค่าเดิม

หมวดที่ 2: DML (Data Manipulation Language) – เพิ่ม แก้ไข ลบข้อมูล

```
-- 1. INSERT ข้อมูลแถวเดียว และหลายแถวพร้อมกัน (Bulk Insert)
INSERT INTO tb_customers (customer_code, first_name, last_name, zone_id)
VALUES
    ('WY-010', 'สมบัติ', 'มีทรัพย์', 1),
    ('WY-011', 'ดวงใจ', 'รักสงบ', 2);

-- 2. INSERT INTO ... SELECT (คัดลอกข้อมูลจากตารางอื่น)
INSERT INTO members_backup SELECT * FROM tb_customers WHERE status = 'ACTIVE';

-- 3. UPDATE ข้อมูล (ระวังลืม WHERE!)
UPDATE meter_readings
SET payment_status = 'PAID', payment_date = CURDATE()
WHERE id = 105;

-- 4. DELETE ข้อมูลตามเงื่อนไข
DELETE FROM service_tickets WHERE status = 'CANCELLED' AND created_at < '2024-01-01';
```

หมวดที่ 3: DQL พื้นฐาน & Operators ครบทุกตัว

3.1 คลังตัวดำเนินการเปรียบเทียบและตรรกะ (Operators Catalog)

ตัวดำเนินการ	ความหมาย	ตัวอย่างการใช้งาน	ผลลัพธ์
=	เท่ากับ	WHERE status = 'PAID'	ดึงแถวที่สถานะตรงกันเป๊ะ
≠ หรือ <>	ไม่เท่ากับ	WHERE zone_id <> 1	ดึงแถวที่โซนไม่ใช่ 1
> / <	มากกว่า / น้อยกว่า	WHERE units_used > 50	ดึงหน่วยใช้น้ำเกิน 50
≥ / ≤	มากกว่าหรือเท่ากับ / น้อยกว่าหรือเท่ากับ	WHERE salary ≥ 30000	ดึงเงินเดือนตั้งแต่ 30,000 ขึ้นไป
BETWEEN a AND b	อยู่ในช่วง a ถึง b (รวมจุดขอบ)	WHERE age BETWEEN 20 AND 30	เทียบเท่ากับ age ≥ 20 AND age ≤ 30
IN (...)	มีค่าตรงกับสมาชิกในชุด	WHERE zone_id IN (1, 2, 4)	เทียบเท่ากับ zone=1 OR zone=2 OR zone=4
LIKE	ค้นหาตามรูปแบบ Wildcard	WHERE name LIKE 'ส%'	ชื่อที่ขึ้นต้นด้วย 'ส'
IS NULL	ตรวจสอบค่าว่าง	WHERE receipt_no IS NULL	ดึงบิลที่ยังไม่ออกใบเสร็จ
IS NOT NULL	ตรวจสอบว่ามีค่า	WHERE phone IS NOT NULL	ดึงเฉพาะคนที่มีเบอร์โทร
AND / OR / NOT	ตรรกะ และ / หรือ / ปฏิเสธ	WHERE (a OR b) AND c	ต้องใส่วงเล็บรอบ OR เสมอ

หมวดที่ 4: ฟังก์ชันเดี่ยวครบวงจร (Built-in Scalar Functions)

4.1 ฟังก์ชันข้อความ (String)

- CONCAT(a, ' ', b) : ต่อข้อความเข้าด้วยกัน
- CONCAT_WS(' ', a, b, c) : ต่อข้อความโดยมีตัวคั่น
- SUBSTRING(str, 1, 5) : ตัดข้อความ (เริ่มนับที่ 1)
- UPPER(s) / LOWER(s) : แปลงเป็นตัวพิมพ์ใหญ่/เล็ก
- TRIM(s) : ตัดช่องว่างส่วนเกินหน้า-หลัง
- REPLACE(str, from, to) : แทนที่ข้อความ
- CHAR_LENGTH(s) : นับจำนวนตัวอักษรจริง
- LPAD(num, 4, '0') : เติมศูนย์ข้างหน้า เช่น '0005'

4.2 ฟังก์ชันตัวเลข & เงื่อนไข (Math & Flow)

- ROUND(val, 2) : ปัดเศษทศนิยม 2 ตำแหน่ง
- CEIL(val) : ปัดเศษขึ้นเป็นจำนวนเต็มเสมอ
- FLOOR(val) : ปัดเศษทิ้งลงเป็นจำนวนเต็มเสมอ
- ABS(val) : หาค่าสัมบูรณ์
- MOD(a, b) : ทหารเอาเศษ (Modulo)
- IF(cond, true_val, false_val) : เงื่อนไข 2 ทางเลือก
- IFNULL(val, default) : ถ้าเป็น NULL ให้ใช้ค่า default
- COALESCE(a, b, c) : คืนค่าตัวแรกที่ไม่ใช่ NULL

4.3 ฟังก์ชันวันและเวลา (Date & Time Functions – จุดขีดาคะแนบ)

ฟังก์ชัน	ตัวอย่างคำสั่ง	ผลลัพธ์ / การนำไปใช้งานจริง
CURDATE() / NOW()	SELECT CURDATE(), NOW();	ดึงวันที่ปัจจุบัน ('2026-10-10') และเวลาที่พร้อมเวลา
YEAR() / MONTH() / DAY()	SELECT YEAR(hire_date);	ดึงชิ้นส่วนปี เดือน หรือ วัน ของวันที่ออกมาเป็นตัวเลข
TIMESTAMPDIFF(unit, d1, d2)	TIMESTAMPDIFF(YEAR, hire_date, CURDATE())	คำนวณอายุงานเต็มปีที่แม่นยำที่สุด (ยังไม่ถึงวันครบรอบไม่ปัดขึ้น)
DATEDIFF(d1, d2)	DATEDIFF('2024-08-30', '2024-08-20')	คำนวณจำนวนวันที่ต่างกัน คำนวณเป็นตัวเลขวัน (ได้ 10)
DATE_ADD() / DATE_SUB()	DATE_ADD(CURDATE(), INTERVAL 15 DAY)	บวก/ลบวันที่ (เช่น หาวันครบกำหนดชำระบิลอีก 15 วันข้างหน้า)
DATE_FORMAT(date, format)	DATE_FORMAT(reading_date, '%d/%m/%Y')	จัดรูปแบบวันที่ไทย เช่น '27/08/2567'

หมวดที่ 5 & 6: การจัดกลุ่ม (Grouping) และการเชื่อมตาราง (JOINS)

5.1 กฎหลักของ GROUP BY & HAVING

- กฎเหล็ก: คอลัมน์ที่ไม่ได้อยู่ใน Aggregate Function (SUM, AVG ฯลฯ) ต้องใส่ใน GROUP BY ทั้งหมด
- WHERE vs HAVING:
 - WHERE กรองแถวก่อนรวมกลุ่ม (ห้ามใส่ SUM/AVG)
 - HAVING กรองกลุ่มหลังคำนวณสถิติเสร็จแล้ว

```
SELECT zone_id, SUM(units_used) AS total_units
FROM meter_readings
WHERE payment_status = 'PAID'
GROUP BY zone_id
HAVING SUM(units_used) > 100;
```

6.1 สรุปชนิดของ Relational JOINS

- INNER JOIN: เอาเฉพาะแถวที่คีย์ตรงกันทั้งสองฝั่ง (\$A \cap B\$)
- LEFT JOIN: เอาฝั่งซ้ายครบทุกแถว ฝั่งขวาไม่มีใส่ NULL
- RIGHT JOIN: เอาฝั่งขวาครบทุกแถว (กลับทิศ LEFT JOIN)
- CROSS JOIN: Cartesian Product จับคู่ทุกแถว (\$M \times N\$)
- SELF JOIN: ตารางเดิม JOIN กับตัวเอง (เช่น พนักงาน-หัวหน้า)
- Anti-Join: LEFT JOIN ... WHERE right.id IS NULL

หมวดที่ 7, 8 & 9: Subqueries, CTEs, Window Functions & Set Operations

7.1 Set Operations (รวมเซต)

- UNION : รวม 2 ตาราง + ตัดแถวซ้ำออก
- UNION ALL : รวม 2 ตาราง ไม่ตัดแถวซ้ำ (เร็วที่สุด)
- INTERSECT : เอาเฉพาะแถวที่เหมือนกันทั้งสองชุด
- EXCEPT / MINUS : เอาชุดแรก หักชุดสองออก

8.1 ชนิดของ Subqueries

- Scalar: คืนค่าเดี่ยวๆ เช่น WHERE salary > (SELECT AVG(...))
- IN / NOT IN: คืนรายการหลายแถว
- EXISTS: ตรวจสอบว่ามีข้อมูลอย่างน้อย 1 แถวหรือไม่ (เร็วกว่า IN)
- Correlated: ซับคิวรีที่ผูกกับแถวด้านนอก รันทีละแถว

9.1 Window Functions: โครงสร้างและการจัดอันดับ

```
-- ไวยากรณ์หลัก:  
FUNCTION() OVER (PARTITION BY group_col ORDER BY sort_col)  
  
-- ตัวอย่างการจัดอันดับ Top 3:  
WITH Ranked AS (  
    SELECT fullname, zone_id, units_used,  
           DENSE_RANK() OVER (PARTITION BY zone_id ORDER BY units_used DESC) AS rk  
    FROM meter_readings  
)  
SELECT * FROM Ranked WHERE rk <= 3;
```

- | | | | |
|--|---|---|----------------------------------|
| • ROW_NUMBER(): 1, 2, 3, 4 (ไม่สนใจค่าซ้ำ) | • RANK(): 1, 2, 2, 4 (ซ้ำอันดับเท่ากัน แต่ลำดับถัดไปกระโดด) | • DENSE_RANK(): 1, 2, 2, 3 (ซ้ำอันดับเท่ากัน ลำดับถัดไปไม่กระโดด) | • LAG(col, 1): คืนค่าแถวก่อนหน้า |
|--|---|---|----------------------------------|

หมวดที่ 12: คลังโจทย์ข้อสอบจริง 25 ข้อ พร้อมเฉลยละเอียดแบบ Step-by-Step

ข้อ 1: SELECT และคำนวณสูตรคำนวณ (Expressions)

พื้นฐาน

จงแสดงรหัสลูกค้า, ชื่อ-นามสกุล, และค่าน้ำดิบที่คำนวณจากหน่วยใช้น้ำคูณหน่วยละ 7 บาท โดยตั้งชื่อคอลัมน์ว่า `calculated_charge`

```
SELECT customer_code, fullname, (units_used * 7.00) AS calculated_charge FROM  
meter_readings;
```

ข้อ 2: กรองข้อมูลตัวเลขด้วย BETWEEN

พื้นฐาน

จงหารายการจดมิเตอร์น้ำที่มียอดใช้น้ำอยู่ระหว่าง 15 ถึง 30 หน่วย ในรอบบิล '8-2567'

```
SELECT * FROM meter_readings WHERE billing_cycle_id = '8-2567' AND units_used BETWEEN  
15.00 AND 30.00;
```

ข้อ 3: ตรรกะร่วม AND / OR และการใส่วงเล็บ

ปานกลาง

จงแสดงสมาชิกในโซน 1 หรือ 2 ที่มีสถานะ 'ACTIVE' และติดตั้งมิเตอร์ขนาด 'มิเตอร์ขนาด 5/8 นิ้ว'

```
SELECT fullname, zone_id, meter_size, status FROM customers
WHERE (zone_id = 1 OR zone_id = 2) AND status = 'ACTIVE' AND meter_size = 'มิเตอร์ขนาด 5/8 นิ้ว';
```

ข้อ 4: การค้นหาข้อความด้วย LIKE Wildcards

ปานกลาง

จงหาข้อร้องเรียนแจ้งซ่อม ('service_tickets') ที่มีคำว่า 'แตก' หรือ 'รั่ว' อยู่ในประเภทปัญหา

```
SELECT ticket_no, reporter_name, issue_type, description FROM service_tickets
WHERE issue_type LIKE '%แตก%' OR issue_type LIKE '%รั่ว%';
```

ข้อ 5: การตรวจจับค่าว่างตามหลัก 3VL ด้วย IS NULL

ปานกลาง

จงแสดงรายการบิลค่าน้ำทั้งหมดที่ยังไม่ได้ออกเลขที่ใบเสร็จรับเงิน ('receipt_no IS NULL')

```
SELECT id, customer_id, grand_total, payment_status FROM meter_readings WHERE
receipt_no IS NULL;
```

ข้อ 6: คำนวณอายุงานจากปีคงที่ด้วย YEAR()

ปานกลาง

จงแสดงชื่อลูกค้า วันที่ติดตั้งมิเตอร์ และจำนวนปีที่ติดตั้งจนถึงปี 2024 เฉพาะผู้ที่ติดตั้งมาแล้ว ≥ 3 ปี

```
SELECT fullname, meter_installed_date, (2024 - YEAR(meter_installed_date)) AS
installed_years
FROM customers WHERE (2024 - YEAR(meter_installed_date))  $\geq$  3;
```

ข้อ 7: คำนวณอายุงานเทียบกับวันปัจจุบันด้วย TIMESTAMPDIFF()

ปานกลาง

จงแสดงรหัสเจ้าหน้าที่ ชื่อ และอายุงานเต็มปีเทียบกับวันปัจจุบันของระบบ เรียงจากมากไปน้อย

```
SELECT user_id, fullname, TIMESTAMPDIFF(YEAR, created_at, CURDATE()) AS service_years
FROM tb_users WHERE role = 'staff' ORDER BY service_years DESC;
```

ข้อ 8: การหาผลรวมและจัดกลุ่มด้วย GROUP BY

ปานกลาง

จงสรุปปริมาณการใช้น้ำรวม ('total_units') และค่าน้ำรวม แยกตามแต่ละโซน

```
SELECT c.zone_id, SUM(mr.units_used) AS total_units, SUM(mr.current_total) AS
total_revenue
FROM meter_readings mr JOIN tb_customers c ON mr.customer_id = c.cus_id GROUP BY
c.zone_id;
```

ข้อ 9: การกรองผลรวมของกลุ่มด้วย HAVING

ปานกลาง

จงหาโซนที่มีลูกหนี้ค้างชำระรวมกันตั้งแต่ 3 รายขึ้นไป

```
SELECT c.zone_id, COUNT(mr.id) AS unpaid_count
FROM meter_readings mr JOIN tb_customers c ON mr.customer_id = c.cus_id
WHERE mr.payment_status = 'UNPAID' GROUP BY c.zone_id HAVING COUNT(mr.id)  $\geq$  3;
```

ข้อ 10: เชื่อมโยง 3 ตารางด้วย INNER JOIN

ปานกลาง

จงแสดงรหัสบิล, ชื่อลูกค้า, ชื่อโซน ('zone_name'), และยอดชำระสุทธิ ('grand_total')

```
SELECT mr.id AS bill_id, c.fullname, z.zone_name, mr.grand_total
FROM meter_readings mr
JOIN tb_customers c ON mr.customer_id = c.cus_id
JOIN tb_zone z ON c.zone_id = z.zone_id;
```

ข้อ 11: Anti-Join ด้วย LEFT JOIN หากไม่มีคู่

ขั้นสูง

จงหารายชื่อลูกค้าที่ไม่มีประวัติการจดมิเตอร์ในรอบบิลปัจจุบันเลย

```
SELECT c.cus_id, c.fullname FROM tb_customers c
LEFT JOIN meter_readings mr ON c.cus_id = mr.customer_id AND mr.billing_cycle_id = 1
WHERE mr.id IS NULL;
```

ข้อ 12: การจำแนกข้อมูลด้วย CASE WHEN

ขั้นสูง

จงแสดงชื่อลูกค้า หน่วยใช้น้ำ และระดับการใช้: <10 'น้อย', 10-25 'ปานกลาง', เกิน 25 'สูง'

```
SELECT c.fullname, mr.units_used,
CASE
    WHEN mr.units_used < 10 THEN 'น้อย'
    WHEN mr.units_used BETWEEN 10 AND 25 THEN 'ปานกลาง'
    ELSE 'สูง'
END AS consumption_level
FROM meter_readings mr JOIN tb_customers c ON mr.customer_id = c.cus_id;
```

ข้อ 13: Conditional Aggregation (Pivot ยอดบิล)

ขั้นสูง

จงสรุปจำนวนบิลในแต่ละโซน แยกเป็นบิลจ่ายแล้ว ('paid_bills') และบิลค้างชำระ ('unpaid_bills')

```
SELECT c.zone_id,
COUNT(CASE WHEN mr.payment_status = 'PAID' THEN 1 END) AS paid_bills,
COUNT(CASE WHEN mr.payment_status = 'UNPAID' THEN 1 END) AS unpaid_bills
FROM meter_readings mr JOIN tb_customers c ON mr.customer_id = c.cus_id GROUP BY
c.zone_id;
```

ข้อ 14: Subquery เกี่ยวกับค่าสูงสุด MAX()

ขั้นสูง

จงหาข้อมูลบิลค่าน้ำที่มียอดใช้น้ำสูงที่สุดในระบบ

```
SELECT * FROM meter_readings WHERE units_used = (SELECT MAX(units_used) FROM
meter_readings);
```

ข้อ 15: Subquery ด้วย NOT IN และเช็ค NULL

ขั้นสูง

จงหารายชื่อลูกค้าที่ไม่มีหนี้ค้างชำระเลยแม้แต่บาทเดียว

```
SELECT cus_id, fullname FROM customers
WHERE cus_id NOT IN (
    SELECT customer_id FROM meter_readings WHERE previous_arrears > 0 AND customer_id
IS NOT NULL
);
```

ข้อ 16: การหายอดเงินสูงสุดอันดับที่ 2 (Second Highest)

ขั้นสูง

จงหายอดเงินค่าน้ำที่สูงที่สุดเป็นอันดับที่ 2 ของหมู่บ้าน

```
SELECT MAX(grand_total) AS second_highest_bill
FROM meter_readings WHERE grand_total < (SELECT MAX(grand_total) FROM meter_readings);
```

ข้อ 17: Subquery ใน FROM (Derived Table)

ขั้นสูง

จงหาค่าเฉลี่ยของ "ยอดเงินรวมต่อโซน" ของทุกโซนในหมู่บ้าน

```
SELECT AVG(zone_total) AS avg_zone_revenue
FROM (
    SELECT c.zone_id, SUM(mr.grand_total) AS zone_total
    FROM meter_readings mr JOIN tb_customers c ON mr.customer_id = c.cus_id GROUP BY
    c.zone_id
) AS ZoneSummary;
```

ข้อ 18: การจัดอันดับด้วย DENSE_RANK()

ระดับเขียน

จงจัดอันดับผู้ที่ใช้น้ำมากที่สุด Top 3 ในแต่ละโซน

```
WITH Ranked AS (
    SELECT c.zone_id, c.fullname, mr.units_used,
           DENSE_RANK() OVER (PARTITION BY c.zone_id ORDER BY mr.units_used DESC) AS rk
    FROM meter_readings mr JOIN tb_customers c ON mr.customer_id = c.cus_id
)
SELECT * FROM Ranked WHERE rk ≤ 3;
```

ข้อ 19: LAG() ฟังก์ชันดึงค่าแถวก่อนหน้า

ระดับเขียน

จงแสดงประวัติการใช้น้ำ พร้อมดึงยอดใช้น้ำเดือนก่อนหน้ามาเปรียบเทียบผลต่าง

```
SELECT customer_id, reading_date, units_used,
       LAG(units_used, 1, 0) OVER (PARTITION BY customer_id ORDER BY reading_date ASC)
AS prev_units,
       (units_used - LAG(units_used, 1, 0) OVER (PARTITION BY customer_id ORDER BY
reading_date ASC)) AS diff
FROM meter_readings;
```

ข้อ 20: Running Total ยอดเงินสะสมต่อเนื่อง

ระดับเขียน

จงแสดงยอดเงินรับชำระรายวัน พร้อมยอดสะสมรวมตั้งแต่วันแรกจนถึงปัจจุบัน

```
SELECT payment_date, SUM(amount_paid) AS daily_income,
       SUM(SUM(amount_paid)) OVER (ORDER BY payment_date ASC) AS running_total
FROM meter_readings WHERE payment_status = 'PAID' AND payment_date IS NOT NULL GROUP BY
payment_date;
```

ข้อ 21: รวบรวมรายการเบิกจ่ายด้วย UNION ALL

ระดับเขียน

จงรวมรายการฎีกาเบิกจ่าย (`payment\_vouchers`) และค่าซ่อมท่อในคำร้อง เป็นบัญชีรายจ่ายแผ่นเดียว

```
SELECT voucher_date AS exp_date, voucher_type AS exp_type, amount, recipient_name AS payee FROM payment_vouchers
UNION ALL
SELECT DATE(resolved_at) AS exp_date, issue_type AS exp_type, repair_cost AS amount, reporter_name AS payee
FROM service_tickets WHERE status = 'RESOLVED' AND repair_cost > 0 ORDER BY exp_date DESC;
```

ข้อ 22: Relational Division ด้วย NOT EXISTS

บอสใหญ่

จงหารายชื่อลูกค้าที่จ่ายค่าน้ำครบทุกรอบบิลที่มีในระบบ (ไม่เคยค้างชำระแม้แต่งวดเดียว)

```
SELECT c.cus_id, c.fullname FROM tb_customers c
WHERE NOT EXISTS (
  SELECT b.id FROM billing_cycles b
  WHERE NOT EXISTS (
    SELECT 1 FROM meter_readings mr
    WHERE mr.billing_cycle_id = b.id AND mr.customer_id = c.cus_id AND
    mr.payment_status = 'PAID'
  )
);
```

ข้อ 23: Multi-table UPDATE with JOIN

บอสใหญ่

จงอัปเดตสถานะบิลเป็น 'PAID' พร้อมใส่วันที่และเลขที่ใบเสร็จจากตารางทำการรายการพร้อมเพย์

```
UPDATE meter_readings mr
JOIN promptpay_transactions pt ON mr.id = pt.bill_id
SET mr.payment_status = 'PAID', mr.payment_date = pt.transaction_date, mr.receipt_no = pt.ref_no
WHERE pt.status = 'SUCCESS' AND mr.payment_status = 'UNPAID';
```

ข้อ 24: SARGable Optimization (เขียนให้เร็วระดับแสง)

บอสใหญ่

เขียนคำสั่งค้นหาบิลในปี 2024 ให้ใช้ B-Tree Index ที่คอลัมน์ reading_date ได้เต็ม 100%

```
SELECT * FROM meter_readings
WHERE reading_date ≥ '2024-01-01' AND reading_date < '2025-01-01';
```

ข้อ 25: Recursive CTE ผังองค์กรไม่จำกัดระดับ (Final Boss)

บอสใหญ่

จงแสดงสายการบังคับบัญชาของเจ้าหน้าที่การประปา จากประธานกองทุน (id=1) ลงไปหาลูกน้องทุกคน

```
WITH RECURSIVE StaffTree AS (  
    SELECT user_id, fullname, manager_id, 1 AS lvl, CAST(fullname AS CHAR(500)) AS  
chain  
    FROM tb_users WHERE user_id = 1  
    UNION ALL  
    SELECT u.user_id, u.fullname, u.manager_id, st.lvl + 1, CONCAT(st.chain, ' → ',  
u.fullname)  
    FROM tb_users u JOIN StaffTree st ON u.manager_id = st.user_id  
)  
SELECT * FROM StaffTree ORDER BY lvl, user_id;
```